Azure Key Vault – Complete Notes

1. What is Azure Key Vault?

Azure Key Vault is an Azure service used to **securely store and manage:**

1. **Secrets**
2. **Keys**
3. **Certificates**

The main purpose of Azure Key Vault is to **keep sensitive information secure and prevent unauthorized access**.

Simple Example

Suppose an ASP.NET Core application needs to connect to a SQL Server.

Normally, we might store the connection string in:

- `appsettings.json`
- `web.config`
- Environment variables

But a connection string may contain sensitive information such as:

- Username
- Password
- Server details

Instead of storing this sensitive information directly inside the application, we can store it as a **Secret in Azure Key Vault**.

---

2. Why Do We Need Azure Key Vault?

Imagine we have an ASP.NET Core Web API:

```
ASP.NET Core Web API
        |
        | Connection String
        ↓
    SQL Database
```

Normally, the connection string may be stored in:

```
appsettings.json
```

For example:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=...;Database=...;User Id=...;Password=..."
  }
}
```

Problem

Anyone who gets access to the configuration file may potentially see the connection string.

For example:

```
Developer
     |
     ↓
appsettings.json
     |
     ↓
Connection String
```

This creates a security risk.

Solution

Store the connection string inside **Azure Key Vault** instead.

```
ASP.NET Core Web API
          |
          | Request Secret
          ↓
 Azure Key Vault
          |
          | Connection String
          ↓
    SQL Database
```

Now the application gets the secret from Key Vault instead of keeping the sensitive value directly in its configuration.

---

3. Important Concept: Authentication vs Authorization

When using Azure Key Vault, we need to control **who or what can access the vault**.

For example:

```
Azure Web App
       |
       | Permission
       ↓
Azure Key Vault
```

We can give the application permission to access a particular secret.

At the same time, another user may not have permission to read that secret.

Therefore:

```
Application → Has permission → Can access secret

Unauthorized User → No permission → Cannot access secret
```

This is one of the major security benefits of Azure Key Vault.

---

4. What is a Secret?

A **Secret** is sensitive information that an application needs but should not be exposed publicly.

Common examples

- Database connection strings
- Passwords
- API keys
- Access tokens
- Client secrets

- Storage account connection strings

Example

Instead of:

```
{
  "ConnectionString": "Server=myserver;Password=MyPassword123"
}
```

We can store:

```
Secret Name:
SQLConnectionString

Secret Value:
Server=myserver;Password=MyPassword123
```

The application retrieves the secret from Key Vault when required.

---

5. What is a Key?

A **Key** in Azure Key Vault is primarily used for **cryptographic operations**.

Cryptographic operations include:

- Encryption
- Decryption
- Signing
- Verification

Simple Example

Suppose we have some sensitive data:

```
Original Data
     |
     | Encryption Key
     ↓
Encrypted Data
```

To decrypt the data:

```
Encrypted Data
     |
     | Encryption Key
     ↓
Original Data
```

The cryptographic key is therefore extremely important and must be protected.

Azure Key Vault can securely manage these keys.

---

6. Example of Using a Key

Suppose we have an Azure Storage Account.

We want to encrypt the data stored in the Storage Account.

```
Azure Storage Account
        |
        | Encryption
        ↓
     Data
```

Encryption requires a cryptographic key.

Instead of managing that key ourselves, we can manage the key using Azure Key Vault.

```
Azure Storage Account
        |
        | Uses Key
        ↓
 Azure Key Vault
        |
        ↓
 Encryption Key
```

The required Azure service can be given permission to use the key.

---

7. Why Should We Store Keys Securely?

Imagine that we encrypt a server's data using a key.

```
Server Data
     |
     | Encryption Key
     ↓
Encrypted Data
```

Now imagine someone accidentally:

- Deletes the key
- Changes the key
- Loses the key
- Gets unauthorized access to the key

This can cause serious problems because the encrypted data may no longer be accessible or usable as expected.

Therefore, cryptographic keys need strong protection and controlled access.

Azure Key Vault helps us securely manage these keys.

---

8. Secret vs Key

This is an important interview question.

Secret	Key
Stores sensitive values	Used for cryptographic operations
Used to store passwords, connection strings, API keys, etc.	Used for encryption, decryption, signing, etc.
Application generally retrieves the value	Applications/services can use the key for cryptographic operations
Example: SQL connection string	Example: encryption key

Easy way to remember

Secret = Sensitive information

Key = Cryptographic operations

---

9. What are Certificates?

Azure Key Vault can also store and manage **digital certificates**.

Certificates are commonly used for:

- HTTPS/TLS
- Authentication
- Establishing trust
- Secure communication

So remember:

```
Azure Key Vault
      |
      +---- Secrets
      |
      +---- Keys
      |
      +---- Certificates
```

---

10. Azure Key Vault Architecture – Simple Example

Consider an ASP.NET Core application connecting to SQL Server.

Without Key Vault

```
             User
              |
              ↓
      ASP.NET Core App
              |
              ↓
      appsettings.json
              |
              ↓
      SQL Connection String
              |
```

```
          ↓
     SQL Database
```

The sensitive connection string is stored with the application configuration.

With Key Vault

```
              User
               |
               ↓
       ASP.NET Core App
               |
               | Request Secret
               ↓
        Azure Key Vault
               |
               | SQL Connection String
               ↓
         SQL Database
```

The sensitive value is kept in Key Vault instead of directly inside the application configuration.

---

11. Key Security Principle

A very important concept is:

> **Give applications/services only the permissions they actually need.**

For example:

```
Azure Web App
      |
      | Read Secret permission
      ↓
Azure Key Vault
```

But:

```
Developer/User
      |
      | No permission
```

```
     ↓
Azure Key Vault
```

This follows the **principle of least privilege**.

---

12. Real-Time Example

Imagine an e-commerce application.

The application needs:

- SQL Database connection string
- Payment API key
- Storage connection string
- Encryption key

Instead of storing everything inside:

```
appsettings.json
```

we can use:

```
                    Azure Key Vault
                  /        |
                 /         |
        SQL Secret    API Secret    Storage Secret
                                         |
                                Encryption Key
```

The application gets permission to access only the required resources.

---

13. Benefits of Azure Key Vault

1. Secure storage

Sensitive information is stored securely instead of being hardcoded into applications.

2. Centralized secret management

Secrets, keys, and certificates can be managed from one centralized service.

3. Access control

We can control which users, applications, and Azure services can access the resources.

4. Reduced exposure

Sensitive values don't need to be directly stored in source code or configuration files.

5. Key management

Cryptographic keys can be securely managed for encryption and other cryptographic operations.

6. Certificate management

Certificates can also be stored and managed.

---

14. Important Interview Questions

Q1. What is Azure Key Vault?

Answer:

Azure Key Vault is an Azure service used to securely store and manage **secrets, cryptographic keys, and certificates**. It helps applications protect sensitive information and provides controlled access to these resources.

Q2. Why do we use Azure Key Vault?

We use Azure Key Vault to avoid storing sensitive information such as:

- Passwords
- Connection strings
- API keys
- Certificates
- Encryption keys

directly in application code or configuration files.

It also provides centralized management and access control.

Q3. What is a Secret in Azure Key Vault?

A Secret is sensitive information stored securely in Key Vault.

Examples:

```
Database Password
Connection String
API Key
Client Secret
```

Q4. What is a Key in Azure Key Vault?

A Key is a cryptographic key used for operations such as:

```
Encryption
Decryption
Signing
Verification
```

Q5. What is the difference between a Secret and a Key?

Secret: Stores sensitive values such as passwords, connection strings, and API keys.

Key: Used for cryptographic operations such as encryption and decryption.

Q6. Can Azure Key Vault store certificates?

Yes.

Azure Key Vault supports:

- Secrets
- Keys
- Certificates

Q7. How does an application access a secret?

The application is given the required permissions to access the Key Vault and retrieve the secret.

For example:

```
Azure Web App
      |
      | Authorized Access
      ↓
Azure Key Vault
      |
      ↓
Secret
```

---

15. Quick Revision

Remember this:

```
Azure Key Vault
       |
       +----------------+
       |                |
    Secrets           Keys
       |                |
 Passwords          Encryption
 API Keys           Decryption
 Connection         Signing
 Strings            Verification
       |
       +----------------+
              |
        Certificates
              |
        HTTPS / TLS
```

One-line definition

Azure Key Vault securely stores and manages secrets, cryptographic keys, and certificates while providing controlled access to them.

Easy memory trick

Secret → Store sensitive values

Key → Perform cryptographic operations

Certificate → Secure identity/communication

Interview-ready answer

> "We use Azure Key Vault to securely manage sensitive information such as connection strings, passwords, API keys, encryption keys, and certificates. Instead of hardcoding these values in application code or configuration files, we store them in Key Vault and provide authorized applications or services with the required permissions to access them."